DART PROGRAMMING CLASS 04

Control Flow & Collections

Control Flow for • while • do-while break • continue	Collections List • Set • Map Collection Operations	Iteration & Logic for-in • forEach Basic Logical Problems

Class Purpose

This handout is designed as a complete teaching guide for Class 04. Students will learn how to control program flow, repeat tasks with loops, stop or skip iterations, and work with Dart collections using practical examples.

Level: Beginner Language: Dart Format: Theory + Code + Practice

01. Learning Objectives

- Understand why loops are used in programming.
- Write and explain the structure of for, while, and do-while loops.
- Use break to stop a loop and continue to skip an iteration.
- Solve beginner-level logical problems using loops and conditions.
- Understand List, Set, and Map and choose the correct collection.
- Perform common collection operations such as add, remove, search, sort, filter, and transform.
- Iterate through List, Set, and Map using different techniques.

By the end of this class, students should be able to write small Dart programs without copying the complete solution from the examples.

02. Control Flow — The Big Picture

Control flow determines the order in which statements are executed. In beginner Dart programming, the most important control-flow tools in this class are loops, break, and continue.

Tool	Purpose	Typical Use
for	Repeat a block a known number of times	Counting, tables, indexes
while	Repeat while a condition is true	Condition-based repetition
do-while	Run first, then check the condition	At least one execution
break	Immediately exit the loop	Stop when target is found
continue	Skip current iteration	Ignore selected values

03. for Loop

Use a for loop when the repetition pattern is known or can be controlled with a counter. A for loop is especially common when working with indexes and ranges of numbers.

Structure / Syntax

```
for (initialization; condition; update) {
  // statements to repeat
}
```

Execution order: Initialization → Condition → Body → Update → Condition → ...

Example 1 — Print 1 to 10

```
void main() {
  for (int i = 1; i <= 10; i++) {
    print(i);
  }
}
```

Example 2 — Print even numbers

```
void main() {
  for (int i = 2; i <= 20; i += 2) {
    print(i);
  }
}
```

Example 3 — Iterate through a List

```
void main() {
  List<String> names = ["Rahim", "Karim", "Jamal"];

  for (int i = 0; i < names.length; i++) {
    print(names[i]);
  }
}
```

Key idea: In a List, the first index is 0. Therefore, a common index-based loop is i = 0; i < list.length; i++.

Homework / Practice

1. Print numbers from 1 to 100 using a for loop.
2. Print all odd numbers from 1 to 50.
3. Print the square of every number from 1 to 10.
4. Print the multiplication table of a given number from 1 to 10.

Important Questions

1. Explain the three parts of a for loop.
2. Why is i < list.length commonly used instead of i <= list.length?
3. What is the difference between i++ and i += 2?

04. while Loop

A while loop repeats a block as long as its condition remains true. The condition is checked before the body executes.

Structure / Syntax

```
initialization;

while (condition) {
  // statements to repeat

  update;
}
```

Example — Print 1 to 5

```
void main() {
  int i = 1;

  while (i <= 5) {
    print(i);
    i++;
  }
}
```

Example — Countdown

```
void main() {
  int i = 5;

  while (i >= 1) {
    print(i);
    i--;
  }
}
```

Important: Always make sure the loop variable or another relevant value changes. Otherwise, the condition may never become false and the program can enter an infinite loop.

Homework / Practice

1. Print numbers from 10 down to 1 using while.
2. Calculate the sum of numbers from 1 to 100 using while.
3. Count how many digits are in a positive integer using while.

Important Questions

1. Where should initialization happen in a while loop?
2. Where should the update happen?
3. What is an infinite loop? Give a simple example.

05. do-while Loop

A do-while loop is different because the body executes first and the condition is checked afterward. Therefore, the body runs at least once.

Structure / Syntax

```
initialization;

do {
  // statements to repeat

  update;
} while (condition);
```

Example — Print 1 to 5

```
void main() {
  int i = 1;

  do {
    print(i);
    i++;
  } while (i <= 5);
}
```

Important Example — Condition is false initially

```
void main() {
  int i = 10;

  do {
    print(i);
  } while (i < 5);
}

// Output:
// 10
```

Why? The body runs first. Only after printing 10 does Dart check whether i < 5 is true.

Homework / Practice

1. Print 1 to 10 using do-while.
2. Print 10 to 1 using do-while.
3. Create a do-while loop whose initial condition is false and observe the output.

Important Questions

1. Why does a do-while loop execute at least once?
2. What is the main difference between while and do-while?
3. When might do-while be useful in a real program?

06. break

The break statement immediately terminates the nearest loop. Execution continues with the statement after the loop.

```
void main() {
  for (int i = 1; i <= 10; i++) {
    if (i == 6) {
      break;
    }

    print(i);
  }
}

// Output:
// 1
// 2
// 3
// 4
// 5
```

Common Use

Use break when you have found what you are looking for and there is no reason to continue checking the remaining items.

07. continue

The continue statement skips the remaining statements of the current iteration and moves to the next iteration.

```
void main() {
  for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
      continue;
    }

    print(i);
  }
}

// Output:
// 1
// 3
// 5
// 7
// 9
```

Remember: break = stop the loop. continue = skip the current iteration.

Homework / Practice

1. Print numbers from 1 to 20, but stop the loop when the number reaches 10.
2. Print only odd numbers from 1 to 20 using continue.
3. Find the first multiple of 7 between 1 and 100 and stop the loop using break.

Important Questions

1. What happens immediately after break executes?
2. What happens immediately after continue executes?

3. Explain break and continue with your own example.

08. Basic Logical Problems

Logical problems help students understand how loops, conditions, variables, and operators work together. Try to solve these without looking at a complete solution first.

1. Sum from 1 to N: Find the sum of all numbers from 1 to N.
2. Even or Odd: Check whether a given number is even or odd.
3. Largest of Three: Find the largest among three numbers.
4. Count Even Numbers: Count how many even numbers exist in a List.
5. Reverse a Number: Convert 12345 into 54321 using arithmetic and a loop.
6. Factorial: Calculate 5! = 5 × 4 × 3 × 2 × 1 = 120.
7. Prime Number: Check whether a number is prime.
8. Multiplication Table: Print the table of a number from 1 to 10.
9. Palindrome Number: Check whether 121 is the same forward and backward.
10. Find Maximum in a List: Find the largest number without using a built-in maximum function.

Worked Example — Factorial

```
void main() {
  int n = 5;
  int factorial = 1;

  for (int i = 1; i <= n; i++) {
    factorial *= i;
  }

  print(factorial); // 120
}
```

Worked Example — Even / Odd

```
void main() {
  int number = 17;

  if (number % 2 == 0) {
    print("Even");
  } else {
    print("Odd");
  }
}
```

Homework / Practice

1. Check whether a number is positive, negative, or zero.
2. Find the smallest of three numbers.
3. Count the number of even and odd values in a List.
4. Check whether a number is a palindrome.
5. Print all prime numbers from 1 to 100.

09. List

A List is an ordered collection. Each item has an index, starting from 0. A List can contain duplicate values.

Creating and Accessing a List

```
void main() {
  List<String> fruits = [
    "Apple",
    "Mango",
    "Banana"
  ];

  print(fruits[0]);   // Apple
  print(fruits[2]);   // Banana
  print(fruits.length); // 3
}
```

Common List Operations

```
List<String> fruits = ["Apple", "Mango"];

// Add
fruits.add("Banana");

// Add multiple items
fruits.addAll(["Orange", "Grape"]);

// Insert at an index
fruits.insert(1, "Guava");

// Remove by value
fruits.remove("Mango");

// Remove by index
fruits.removeAt(0);

// Update a value
fruits[0] = "Watermelon";

// Search
print(fruits.contains("Banana"));

// Size
print(fruits.length);
```

Useful List Properties / Methods

length — number of items | contains() — checks whether a value exists | sort() — sorts a List

Important Questions

1. What is the first index of a List?
2. Can a List contain duplicate values?
3. What is the difference between remove() and removeAt()?
4. What does addAll() do?

10. Set

A Set is a collection of unique values. If the same value is added more than once, the Set keeps only one occurrence.

Creating a Set

```
void main() {
  Set<String> skills = {
    "Dart",
    "Flutter",
    "Dart"
  };

  print(skills);
  // Dart appears only once
}
```

Common Set Operations

```
Set<int> a = {1, 2, 3, 4};
Set<int> b = {3, 4, 5, 6};

print(a.union(b));
// {1, 2, 3, 4, 5, 6}

print(a.intersection(b));
// {3, 4}

print(a.difference(b));
// {1, 2}
```

Practical Use — Remove Duplicates

```
List<int> numbers = [1, 2, 2, 3, 3, 4];

Set<int> uniqueNumbers = numbers.toSet();

print(uniqueNumbers);
// {1, 2, 3, 4}
```

Homework / Practice

1. Create a Set of programming languages and add/remove values.
2. Find the common values between two Sets using intersection().
3. Find all unique values from two Sets using union().
4. Convert a List with duplicate values into a unique collection.

Important Questions

1. What is the main difference between List and Set?
2. Why is Set useful for removing duplicates?
3. What do union(), intersection(), and difference() return?

11. Map

A Map stores data as key-value pairs. Keys are used to access their corresponding values.

Creating and Accessing a Map

```
void main() {
  Map<String, dynamic> student = {
    "name": "Rahim",
    "age": 22,
    "isStudent": true,
  };

  print(student["name"]); // Rahim
  print(student["age"]);  // 22
}
```

Map Operations

```
Map<String, dynamic> user = {
  "name": "Karim",
  "age": 25,
};

// Add
user["city"] = "Dhaka";

// Update
user["age"] = 26;

// Remove
user.remove("city");

// Search
print(user.containsKey("name"));
print(user.containsValue(26));

// Keys and values
print(user.keys);
print(user.values);
```

Important: A Map is ideal when data has labels/keys, such as name, age, email, city, price, or status.

Homework / Practice

1. Create a student Map containing name, age, email, city, and marks.
2. Add a new key and update an existing value.
3. Check whether a particular key exists.
4. Remove one key and print the final Map.

Important Questions

1. What is a key-value pair?
2. How do you access a Map value?
3. What is the difference between containsKey() and containsValue()?

12. Collection Operations

Dart collections provide many operations for adding, removing, searching, sorting, filtering, and transforming data.

sort(), contains(), removeWhere()

```
List<int> numbers = [5, 2, 8, 1, 4];

print(numbers.contains(8)); // true

numbers.sort();
print(numbers); // [1, 2, 4, 5, 8]

numbers.removeWhere((n) => n < 4);
print(numbers); // [4, 5, 8]
```

where() — Filter Items

```
List<int> numbers = [1, 2, 3, 4, 5, 6];

List<int> evenNumbers =
    numbers.where((n) => n % 2 == 0).toList();

print(evenNumbers);
// [2, 4, 6]
```

map() — Transform Items

```
List<int> numbers = [1, 2, 3, 4, 5];

List<int> squares =
    numbers.map((n) => n * n).toList();

print(squares);
// [1, 4, 9, 16, 25]
```

Remember: where() selects items that satisfy a condition. map() transforms each item into another value.

Homework / Practice

1. Use where() to get all even numbers from 1 to 20.
2. Use map() to create a List containing the square of each number from 1 to 10.
3. Sort a List of numbers in ascending order.
4. Remove all numbers less than 5 using removeWhere().

13. Iteration

Iteration means processing collection items one by one. Dart provides several ways to iterate through a collection.

Method 1 — Traditional for Loop

```
List<String> names = ["Rahim", "Karim", "Jamal"];

for (int i = 0; i < names.length; i++) {
  print(names[i]);
}
```

Method 2 — for-in Loop

```
for (String name in names) {
  print(name);
}
```

Method 3 — forEach()

```
names.forEach((name) {
  print(name);
});
```

Map Iteration

```
Map<String, dynamic> user = {
  "name": "Rahim",
  "age": 22,
};

user.forEach((key, value) {
  print("$key : $value");
});
```

Which One Should I Use?

Method	Good For
for	When you need the index or more control.
for-in	Simple item-by-item iteration.
forEach()	Short operations using a callback.

Homework / Practice

1. Print every item of a List using for.
2. Print every item of a List using for-in.
3. Print every item of a List using forEach().
4. Iterate through a Map and print each key and value.

Important Questions

1. What is iteration?
2. What is the advantage of for-in over an index-based loop?
3. When would you prefer a traditional for loop?
4. How can you iterate through a Map?

14. Master Comparison — List vs Set vs Map

Feature	List	Set	Map
Data format	Ordered values	Unique values	Key-value pairs
Index	Yes	No index access like List	Access by key
Duplicates	Allowed	Not kept	Keys must be unique
Example	[10, 20, 20]	{10, 20}	{"age": 22}
Best use	Ordered data	Unique data	Labeled data

15. Loop Comparison

Feature	for	while	do-while
Condition checked	Before each iteration	Before each iteration	After each iteration
Minimum executions	0	0	1
Best for	Known repetition	Condition-based repetition	At least one execution

16. Final Homework — Class Assignment

Solve the following problems independently. Do not copy the complete code from the examples. First write the logic on paper, then implement it in Dart.

1. Create a List of student marks. Calculate the total and average.
2. Find the highest and lowest mark in a List without using a built-in maximum/minimum function.
3. Separate even and odd numbers from a List into two different Lists.
4. Remove duplicate values from a List.
5. Create a Map for three students and store their names and marks. Iterate through the Map and print the data.
6. Implement FizzBuzz from 1 to 100: multiples of 3 → Fizz, multiples of 5 → Buzz, multiples of both → FizzBuzz.
7. Check whether a number is prime.
8. Check whether a number is a palindrome.
9. Use break to stop a search after finding the first matching value.
10. Use continue to skip values that do not meet a condition.
11. Use where() to filter a List and map() to transform the filtered values.

17. Important Viva / Class Questions

1. What is a loop and why do we use it?
2. Write the complete structure of a for loop and explain each part.
3. What is the difference between for and while?
4. What is the difference between while and do-while?
5. Why does do-while execute at least once?
6. What is an infinite loop and how can you avoid it?
7. What does break do?
8. What does continue do?
9. What is the difference between List, Set, and Map?
10. Can a List contain duplicate values?
11. Why is Set useful when working with duplicate data?
12. What is a key-value pair in a Map?
13. How do you add, update, and remove data from a Map?
14. What is the difference between where() and map()?
15. What are three different ways to iterate through a List?
16. How do you iterate through a Map's keys and values?

18. Class Checklist

Concept	Student can explain	Student can code
for loop	☐	☐
while loop	☐	☐

Concept	Student can explain	Student can code
do-while loop	☐	☐
break / continue	☐	☐
Basic logical problems	☐	☐
List	☐	☐
Set	☐	☐
Map	☐	☐
Collection operations	☐	☐
Iteration	☐	☐

Teacher Tip: Ask students to predict the output before running each example. This makes loop execution and collection behavior much easier to understand.